

КИЇВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ
ТАРАСА ШЕВЧЕНКА
МЕХАНІКО-МАТЕМАТИЧНИЙ ФАКУЛЬТЕТ
КАФЕДРА ТЕОРІЇ ЙМОВІРНОСТЕЙ, СТАТИСТИКИ ТА
АКТУАРНОЇ МАТЕМАТИКИ

КУРСОВА РОБОТА

НА ТЕМУ:

Сегментація експериментальних кривих із застосуванням динамічного
програмування

Виконала:

студентка 4 курсу

механіко-математичного факультету

групи «Статистика»

Шкляр Ірина Володимирівна

Науковий керівник:

доктор фізико-математичних наук

професор кафедри теорії

ймовірностей,

статистики та актуарної математики

Майборода Ростислав Євгенович

Зміст

Вступ.....	2
1. Теоретична частина.....	3
1. Логіка та алгоритм сегментації.....	3
1.1. Ідея сегментації часових рядів.....	3
1.2. Застосування та опис сегментації в конкретному випадку цієї дипломної роботи.....	4
2. Логіка динамічного програмування.....	7
2.1. Загальний опис.....	7
2.2. Ідея динамічного програмування.....	9
2.3. Оптимальність для підзадач.....	12
2.4. Динамічне програмування в методах оптимізації.....	13
2.5. Застосування та опис динамічного програмування в конкретному випадку цієї дипломної роботи.....	13
2. Практична частина.....	17
1. Програмне забезпечення.....	17
1.1. Використані технології.....	17
1.2. Опис та збір даних.....	17
2. Опис алгоритму.....	18
3. Результати та їх аналіз.....	22
Висновки.....	33
Список використаних джерел.....	34
Додаток 1 (код програми).....	37

Вступ

Аналіз експериментальних кривих, представлених у вигляді часових рядів, є важливим завданням у багатьох сферах науки та інженерії. Дуже часто поведінка даних не є однорідною на всьому проміжку, а демонструє різні тенденції – спади, стрибки, зростання тощо, на окремих сегментах. Виявлення цих сегментів може показати певні закономірності чи особливості процесу, для якого описані дані.

Задача сегментації експериментальних кривих полягає в розбитті вихідного часового ряду на декілька сегментів таким чином, щоб дані в межах кожного сегмента були більш менш однорідними, тобто задовольняли певний критерій однорідності, в той час як суміжні сегменти відрізнялися один від одного.

У роботі ми розглянемо проблему сегментації, і як можна сегментувати експериментальні криві та часові ряди, яким притаманна лінійна регресія, за допомогою алгоритмів динамічного програмування. Зробимо так, щоб наш алгоритм дозволяв знаходити оптимальні розбиття на сегменти за допомогою мінімізації певної функції — вартості сегментації або сумарної похибки.

Важливим аспектом задачі є визначення оптимальної кількості сегментів у розбитті. Для цього в роботі будемо застосовувати байєсівський інформаційний критерій, який дозволяє знайти компроміс між якістю апроксимації даних та складністю моделі (кількістю сегментів).

Розроблені алгоритми реалізовуємо у вигляді коду програми на мові програмування Python з використанням бібліотек NumPy, Pandas та Matplotlib для обробки даних та візуалізації результатів, та програмного середовища PyCharm. Ми протестуємо їх на різних наборах даних, як синтетичних, так і реальних, щоб продемонструвати їх ефективність.

Головна ціль цієї роботи - розробити та програмно реалізувати методи для оптимального розбиття графіків та часових рядів на логічні сегменти для полегшення їх аналізу та інтерпретації.

Розділ 1

Теоретична частина

1. Логіка та алгоритм сегментації

1.1. Ідея сегментації часових рядів

Часовий ряд (англ. *time series*) — це ряд точок даних, проіндексованих (або перелічених, або відкладених на графіку) в хронологічному порядку. Найчастіше часовий ряд є послідовністю, взятою на рівновіддалених точках в часі, які йдуть одна за одною. Таким чином, він є послідовністю даних дискретного часу. Прикладами часових рядів є висоти океанських припливів, коливання погоди, та інше.

Сегментація часових рядів - це метод аналізу часових рядів, в якому вхідний часовий ряд розбивається на послідовність дискретних сегментів, щоб виявити основні властивості його джерела. Типовим застосуванням сегментації часових рядів є діаризація мовлення, коли аудіосигнал розбивається на кілька частин відповідно до того, хто і в який час говорить. Алгоритми, засновані на виявленні точок зміни, включають висхідні та низхідні методи. Імовірнісні методи, засновані на прихованих марківських моделях, також виявилися корисними у вирішенні цієї проблеми. [23], [12]

Огляд проблеми сегментації

Часто буває так, що часовий ряд можна представити як послідовність дискретних відрізків скінченної довжини. Наприклад, траєкторію фондового ринку можна розбити на інтервали, які лежать між важливими світовими подіями, або аудіозапис конференції можна розділити відповідно до того, хто і коли виступав. В останньому випадку можна скористатися тим, що мітки окремих сегментів можуть повторюватися (наприклад, якщо людина виступає кілька разів під час конференції), спробувавши згрупувати сегменти за їхніми відмітними властивостями (наприклад, за

спектральним складом голосу кожного учасника). Існує два загальних підходи до цієї проблеми. Перший передбачає пошук точок зміни в часовому ряді: наприклад, можна визначити межу сегмента, коли спостерігається великий стрибок середнього значення сигналу. Другий підхід передбачає припущення, що кожен сегмент часового ряду генерується системою з певними параметрами, а потім виводить найбільш ймовірні місця розташування сегментів і параметри системи, які їх описують. У той час як перший підхід шукає зміни лише в короткому проміжку часу, другий підхід зазвичай бере до уваги весь часовий ряд, коли вирішує, яку мітку присвоїти певній точці. [23]

Алгоритми сегментації

Приховані марківські моделі (ПММ; Hidden Markov Models, HMM)

Згідно з прихованою марківською моделлю, для часового ряду $y_{1:T} = (y_1, y_2, \dots, y_T)$ припускається, що він генерується при переходах системи між множиною дискретних, прихованих станів $z \in \{1, 2, \dots, n\}$. У кожен момент часу t з розподілу спостережень (або викидів) береться вибірка y_t , індексована поточним прихованим станом, тобто $y_t \sim P_{z_t}(y_t)$. Метою задачі сегментації є визначення прихованого стану в кожен момент часу, а також параметрів, що описують розподіл випромінювання, пов'язаний з кожним прихованим станом. Послідовність прихованих станів і параметри розподілу випромінювання можна дізнатися за допомогою алгоритму Баума-Велча, який є варіантом максимізації математичного сподівання, застосованого до ПММ. Зазвичай, в задачі сегментації, ймовірності самопереходів між станами вважаються високими, так що система залишається в кожному стані протягом невід'ємного часу. [12]

1.2. Застосування та опис сегментації в конкретному випадку цієї дипломної роботи

У цій роботі ми маємо наступну задачу сегментації часового ряду: розбити заданий часовий ряд на декілька сегментів (тобто блоків суміжних даних),

таким чином, щоб кожен сегмент був однорідним, тоді як суміжні сегменти були неоднорідними (однорідність визначається в термінах відповідної статистики сегментів).

Проблема сегментації часових рядів часто виникає в гідрології та енвайронментології (іноді її називають виявленням та оцінкою точок змін). У роботі [20] була розглянута ця проблема, використовуючи формулювання прихованої марківської моделі (ПММ). Зазначимо, що і ця робота, і [20] були мотивовані процедурою сегментації Х'юберта [14], [15] і роботою [11] (де сегментація часових рядів розглядається з використанням моделі зсувних середніх та методів ланцюгових марківських моделей Монте-Карло). [19]

Інтерес до проблем сегментації часових рядів останнім часом зріс у літературі з інтелектуального аналізу даних ([13]; [21] і, особливо, [22], де представлено гарний огляд методів сегментації, що з'являються в літературі з інтелектуального аналізу даних).

Ми розглядаємо сегментацію часових рядів (або інших експериментальних кривих) як обчислювальну задачу, і наше головне завдання полягає у швидкій та ефективній її реалізації. Ми слідуємо (і узагальнюємо) формулюванню Х'юберта про сегментацію як оптимізаційну задачу. Підхід ПММ, запропонований в [20], є досить швидким, але не гарантує отримання глобально оптимальної сегментації.

Сегментація як оптимізація

Формулювання та позначення, представлені тут значною мірою, ґрунтуються на [14], [15]. Сегментацією цілих чисел $\{1, 2, \dots, T\}$, називається послідовність $\mathbf{t} = (t_0, t_1, \dots, t_K)$, яка задовольняє $0 = t_0 < t_1 < \dots < t_{K-1} < t_K = T$.

Інтервали цілих чисел $[t_0 + 1, t_1]$, $[t_1 + 1, t_2]$, $\dots$, $[t_{K-1} + 1, t_K]$

називаються *сегментами*, моменти часу $t_0, t_1, \dots, t_K$ називаються границями сегментів або розривами, а K - кількість сегментів (відрізків), називається *порядком сегментації*. Будемо позначати множину усіх сегментів $\{1, 2, \dots, T\}$ через T , а множину всіх сегментів порядку K - через T_K .

Очевидно, що $T = \bigcup_{K=1}^T T_K$. Позначимо через $\text{card}(\cdot)$ кількість елементів множини, маємо $\text{card}(T_K) = \binom{T-1}{K-1}$ і $\text{card}(T) = \sum_{K=1}^T \text{card}(T_K) = 2^{T-1}$.

У багатьох застосуваннях задано часовий ряд $x_1, x_2, \dots, x_T$ і ми шукаємо сегментацію $\{1, 2, \dots, T\}$, яка відповідає змінам поведінки $x_1, x_2, \dots, x_T$ (у цьому випадку межі сегментів також називаються точками змін (*change points*)). Це можна сформулювати як оптимізаційну задачу. Ми визначимо вартість сегментації $J(t)$ наступним чином:

$$J(t) = \sum_{k=1}^K d_{t_{k-1}+1, t_k}, \quad (1)$$

де $d_{s,t}$ (для $0 \leq s < t \leq T$) – середньоквадратична похибка на сегменті (*segment error*), що відповідає відрізку $[s, t]$. Оптимальна сегментація, що позначається як $\hat{t} = (\hat{t}_0, \hat{t}_1, \dots, \hat{t}_K)$ визначається як $\hat{t} = \arg \min_{t \in T} J(t)$,

і оптимальна сегментація порядку K , що позначається як $\hat{t}^{(K)} = (t_0^{(K)}, t_1^{(K)}, \dots, t_K^{(K)})$ визначається як $\hat{t}^{(K)} = \arg \min_{t \in T_K} J(t)$.

Очевидно, що похибка відрізка $d_{s,t}$ залежить від даних $x_1, x_2, \dots, x_T$. У (1) можна використовувати різні функції $d_{s,t}$. Далі ми представимо декілька таких функцій, кожна з яких матиме вигляд (для $0 \leq s < t \leq T$)

$$d_{s,t} = \sum_{\tau=s}^t (x_\tau - \hat{x}_\tau)^2, \quad (2)$$

де $\hat{x}_\tau$ буде деякою регресійною оцінкою x_τ , отриманою з даних всередині сегмента (*within-segment data*) $x_s, x_{s+1}, \dots, x_t$. Звідси $d_{s,t}$ - це вибіркова дисперсія на відрізку $[s, t]$, помножена на $(t - s)$.

Вибір порядку сегментації

Отримавши оптимальну сегментацію для кожного значення K , залишається вибрати "правильне" значення K , тобто "істинну" кількість сегментів. Це є доволі складною проблемою. Вона є окремим випадком проблеми вибору моделі, яка широко досліджувалась і, очевидно, ще не була розв'язана

задовільно (огляди [8], [18]). Х'юберт вирішує цю проблему шляхом перевірки за допомогою критерію Шеффе нульової гіпотези про те, що послідовні відрізки не мають суттєво відмінних середніх; за істинне значення $\hat{K}$ він бере найбільше значення K , для якого нульова гіпотеза може бути підтверджена для відрізків k і $k+1$ (з $k=1, 2, \dots, K-1$). Цей підхід пов'язаний з деякими труднощами. Деякі з них зазначені в [6]); на думку авторів, основна складність полягає в тому, що послідовні вибірки не є незалежними; крім того, не одразу зрозуміло, як узагальнити критерій Шеффе, коли статистика сегмента не є середньою. [19]

Тому використовуємо інший підхід – байєсівський інформаційний критерій або критерій Шварца (*Bayesian information criterion* – *BIC*) в термінах залишкової суми квадратів (RSS), [27]:

$$BIC = \tilde{J}(K) = T \log(\hat{J}(K)/T) + R_K \log(T) \quad (3)$$

(де R_K - кількість параметрів у моделі K -го порядку, $\hat{J}(K)$ – сумарна похибка або залишкова сума квадратів)) і візьмемо $\hat{K}$ за значення K , яке мінімізує $\tilde{J}(K)$. Загальну мотивацію використання BIC можна знайти в [28]; класичне застосування до задач про точки змін з'являється в [29]. Хоча BIC не завжди дає хорошу оцінку справжньої кількості відрізків, але він працює набагато краще, ніж альтернативні критерії, такі як [1] і [2] та [24].

2. Логіка динамічного програмування

2.1. Загальний опис

Динамічне програмування - це коли у нас є завдання, яке не зрозуміло як вирішувати, і ми розбиваємо його на менші завдання, які теж не зрозуміло як вирішувати. (с) А. Кумок

Динамічне програмування (ДП; Dynamic Programming, DP) — розділ математики, який присвячено теорії та методам розв'язання багатокрокових задач оптимального управління.

Інакше кажучи, це спосіб розв'язання складних задач шляхом розбиття їх на простіші підзадачі. Він може бути застосований до завдань з оптимальною підструктурою, що має вигляд набору підзадач, що перекриваються, складність яких менша за вихідну. У цьому випадку час обчислень, порівняно з "наївними" методами, можна значно скоротити.

У динамічному програмуванні, серед множини усіх допустимих управлінь шукають оптимальне, у сенсі деякого критерію, тобто таке, яке призводить до екстремального (найбільшого або найменшого) значення цільової функції — деякої числової характеристики процесу. Під багатоступеневістю розуміють або багатоступеневу структуру процесу, або розподілення управління на ряд послідовних етапів, що відповідають, як правило, різним моментам часу. Таким чином, в назві «Динамічне програмування» під «програмуванням» розуміють «ухвалення рішень», «планування», а слово «динамічне» вказує на суттєве значення часу та порядку виконання операцій в процесах і методах, що розглядаються.

Це словосполучення – "динамічне програмування", вперше було використане в 1940-х роках Річардом Беллманом для опису процесу знаходження розв'язку задачі, де відповідь на неї можна отримати тільки після розв'язання іншої задачі, яка "передуює" їй. Зазвичай, щоб розв'язати поставлену задачу, потрібно розв'язати окремі частини задачі (підзадачі), після чого об'єднати розв'язки підзадач в одне загальне рішення. Часто, багато з цих підзадач однакові. Підхід динамічного програмування полягає в тому, щоб розв'язати кожну підзадачу тільки один раз, скоротивши тим самим кількість обчислень. Це особливо корисно у випадках, коли кількість повторюваних підзадач експоненційно велика. [3], [10]

Динамічне програмування є водночас і методом математичної оптимізації і методом комп'ютерного програмування. В обох контекстах воно використовує підхід спрощення пошуку розв'язку складної задачі, розбиттям її на простіші підзадачі, часто методом рекурсії. Хоча деякі задачі не можуть бути розв'язані таким чином, рішення, які охоплюють кілька точок у часі дійсно часто розбиваються рекурсивно на підзадачі. Беллман назвав це принципом оптимальності. [9]

Методи динамічного програмування використовуються не лише в дискретних, але і в неперервних керованих процесах, наприклад, в таких процесах, коли в кожен момент певного проміжку часу необхідно ухвалювати рішення.

Хоча метод динамічного програмування суттєво спрощує вихідні задачі, та безпосереднє його використання, як правило, пов'язане з громіздкими обчисленнями. Для подолання цих труднощів розробляються наближені методи динамічного програмування.

Метод *динамічного програмування зверху* - це просте запам'ятовування результатів розв'язання тих підзадач, які можуть повторно зустрітися надалі. *Динамічне програмування знизу* передбачає переформулювання складного завдання у вигляді рекурсивної послідовності простіших підзадач.

2.2. Ідея динамічного програмування

Процес розробки алгоритмів динамічного програмування

У процесі складання алгоритмів динамічного програмування потрібно дотримуватися послідовності з чотирьох дій, [30]:

1. Описати структуру оптимального рішення.
2. Рекурсивно визначити значення оптимального рішення.
3. Обчислити значення оптимального рішення за допомогою методу висхідного аналізу.
4. Скласти оптимальне рішення на основі отриманої інформації.

Оптимальна підструктура в динамічному програмуванні означає, що оптимальне розв'язання підзавдань меншого розміру може бути використане для розв'язання вихідної задачі. Наприклад, найкоротший шлях у графі з однієї вершини (позначимо s) в іншу (позначимо t) може бути знайдений так: спочатку рахуємо найкоротший шлях з усіх вершин, суміжних з s , до t , а потім, з огляду на ваги ребер, якими s сполучено з суміжними вершинами, обираємо найкращий шлях до t (через яку вершину найкраще піти). У загальному

випадку ми можемо розв'язати задачу, в якій присутня оптимальна підструктура, проробляючи такі три кроки:

1. Розбиття завдання на підзадачі меншого розміру.
2. Знаходження оптимального рішення підзадач рекурсивно, проробляючи такий самий трикроковий алгоритм.
3. Використання отриманого рішення підзадач для конструювання рішення вихідної задачі.

Підзадачі розв'язуються діленням їх на підзадачі ще меншого розміру, поки не приходять до тривіального випадку задачі, розв'язуваної за сталий час (відповідь можна сказати одразу). Наприклад, якщо нам потрібно знайти $n!$, то тривіальним завданням буде $1! = 1$ (або $0! = 1$). [3]

Підзадачі, що перекриваються, у динамічному програмуванні означають підзадачі, які використовуються для розв'язання деякої кількості завдань (більше однієї) більшого розміру (тобто ми кілька разів проробляємо одне й те саме). Яскравим прикладом є обчислення послідовності Фібоначчі,

$F_3 = F_2 + F_1$ і $F_4 = F_3 + F_2$ - навіть у такому тривіальному випадку обчислення всього двох чисел Фібоначчі, ми вже порахували F_2 двічі. Якщо продовжувати далі і порахувати F_5 , то F_2 порахується ще два рази, оскільки для обчислення F_5 будуть потрібні знову F_3 і F_4 .

Виходить наступне: простий рекурсивний підхід витратить час на обчислення розв'язку для задач, які він уже розв'язував. [3], [5]

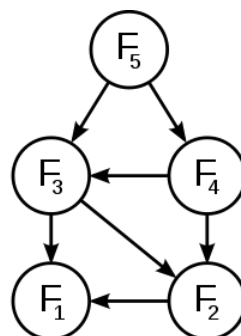


Рис.1: граф підзадач для чисел Фібоначчі (ребро означає, що одне завдання залежить від розв'язання іншого)

Щоб уникнути такого перебігу подій, ми зберігатимемо розв'язки підзадач, які ми вже розв'язували, і коли нам знову знадобиться розв'язок підзадачі, ми замість того, щоб обчислювати його заново, просто дістанемо його з пам'яті. Цей підхід називається мемоїзацією. Можна проробляти і подальші оптимізації - наприклад, якщо ми точно впевнені, що розв'язання підзадачі нам більше не знадобиться, можна викинути її з пам'яті, звільнивши її для інших потреб.

Підбиваючи підсумки вищесказаного, можна сказати, що динамічне програмування користується такими властивостями задачі:

- підзадачі, що перекриваються;
- оптимальна підструктура;
- можливість запам'ятовування розв'язків підзадач, що часто зустрічаються.

Динамічне програмування зазвичай дотримується двох підходів до розв'язання задач:

1. Спадне (низхідне) динамічне програмування (*Динамічне програмування знизу*): задачу розбивають на підзадачі меншого розміру, їх розв'язують і потім комбінують для розв'язання вихідної задачі. Використовується запам'ятовування для рішень уже вирішених підзадач.
2. Висхідне динамічне програмування (*Динамічне програмування зверху*): усі підзадачі, які згодом знадобляться для розв'язання вихідної задачі, прораховуються заздалегідь і потім використовуються для побудови розв'язання вихідної задачі.

Мови програмування можуть запам'ятовувати результат виклику функції з певним набором аргументів (мемоїзація), щоб прискорити "обчислення за іменем".

Однією з основних властивостей задач, розв'язуваних за допомогою динамічного програмування, є адитивність. [3], [5]

2.3. Оптимальність для підзадач

Найважливіша властивість задач, яка дає змогу розв'язувати їх за допомогою динамічного програмування, це оптимальність для підзадач. Залежно від формулювання задачі, чи то динамічне програмування на відрізку, на префіксі, на дереві, термін оптимальності для підзадач може бути різним, але загалом формулюється так: якщо є оптимальний розв'язок для певної підзадачі, що виникає в процесі розв'язання задачі, то саме його потрібно використовувати для розв'язання задачі загалом.

Принцип оптимальності на підвідрізках (підсегментах)

Потрібно порахувати функцію $f(1,n)$. Принцип полягає в наступному: нехай для всіх відрізків i, j (де $u \leq i \leq j \leq v$) відома оптимальна відповідь для функції $f(i, j)$. Тоді ми будемо обчислювати $f(u, v)$ через ці $f(i, j)$. Як приклад розглянемо таку класичну задачу: дано рядок довжини n , потрібно знайти максимальний підпаліндром (підпоследовність максимальної довжини, яка є паліндромом). Нехай $d(i, j)$ - відповідь на задачу для підрядка, що починається із символу i і закінчується в символі j . Зрозуміло, що $d(i, j)=0$ для всіх i, j , таких що $i > j$ і $d(i, i)=1$ для всіх i . Нехай нам потрібно порахувати значення для $d(i, j)$, причому значення d для всіх l, r , таких що $i \leq l \leq r \leq j$ вже пораховані і вони оптимальні. Розглянемо два випадки:

- $s(i) \neq s(j)$, тоді $d(i, j) = \max(d(i, j-1), d(i+1, j))$
- $s(i) = s(j)$, тоді $d(i, j) = d(i+1, j-1) + 2$

Доведення: оскільки $s(i) \neq s(j)$, символи $s(i)$ і $s(j)$ не можуть входити до максимального підпаліндрома одночасно, тобто або $s(i)$ входить до максимального підпаліндрома (тоді його довжина $d[i, j-1]$), або $s(j)$ входить до максимального підпаліндрома (тоді його довжина $d[i+1, j]$), або обидва не входять до максимального підпаліндрома (тоді його довжина $= d[i, j-1] = d[i+1, j]$). Ця рівність впливає з факту, що вигідно включити в максимальний підпаліндром символи $s(i)$ і $s(j)$. [30]

2.4. Динамічне програмування в методах оптимізації

З точки зору математичної оптимізації, динамічне програмування полягає в спрощенні знаходження загального оптимального розв'язку, шляхом пошуку розв'язків в підзадачах, отриманих розбиттям задачі на послідовні проміжки часу. Це виражається у визначенні послідовності значень функцій $V_1, V_2, \dots, V_n$, з аргументом u , котрий позначає **стан** системи в моменти часу i від 1 до n . Визначенням $V_n(u)$ є значення, отримане в стані u в кінцевий момент часу n . Значення V_i в попередні моменти часу $i = n - 1, n - 2, \dots, 2, 1$ можуть бути знайдені рухаючись назад, використовуючи рекурсивну залежність, названу рівнянням Беллмана. Для $i = 2, \dots, n$, значення V_{i-1} для будь-якого стану u визначається з V_i через максимізацію значення простої функції (зазвичай, суми) виграшу від рішення в момент часу $i - 1$ і функції V_i у новому стані системи, якби це рішення було втілено. Оскільки V_i вже було розраховано для потрібних станів, то вище наведена операція забезпечує необхідне оптимальне значення V_{i-1} для цих станів. Нарешті, V_1 як початковий стан системи є значенням оптимального рішення. Оптимальне значення змінних рішення може бути відновлене одне за одним, виконанням обернених у часі обчислень. [9]

2.5. Застосування та опис динамічного програмування в конкретному випадку цієї дипломної роботи

Динамічне програмування часто використовували в минулому для сегментації кривих [4] та форм хвиль ([16], [17], [26]). DP використовується в обчислювальній біології для сегментації послідовностей ДНК ([1], [7], [6]), які застосовують до дискретних послідовностей деякі алгоритми, подібні до того, що представлено тут). У деяких з робіт ([13], [21], [22]) стверджується, що ДП не є обчислювально життєздатним підходом. Ця робота може слугувати частковим спростуванням таких тверджень.

В [20] зазначено, що розв'язок DP дає глобально оптимальну сегментацію ціною більшого часу виконання. У цій роботі ми намагаємося спростити цей

підхід таким чином, щоб він був обчислювально життєздатним для часових рядів з кількома сотнями та тисячами спостережень.

Таким чином, підхід динамічного програмування можна розглядати як покращення процедури Х'юберта для сегментації часових рядів з багатьма точками змін. (Процедура базується на мінімізації вартості сегментації Х'юберта. Але це не є практичним, з обчислювальної точки зору, оскільки загальна кількість сегментацій зростає експоненційно. Т. Х'юберт використовує підхід з розгалуженням для ефективного пошуку множини всіх можливих сегментацій. У [14], [15] зазначено, що цей підхід "...на момент написання статті" (в [15]) може сегментувати часові ряди з кількома десятками членів, але не здатний "...впоратися з рядами з набагато більшою кількістю членів ..." через комбінаторне збільшення обчислювального навантаження. Як буде показано далі, динамічне програмування дозволяє обчислювати ряди з більшою кількістю доданків. Динамічне програмування дозволяє обчислити оптимальну сегментацію за час $O(T^2)$ і дає за кілька секунд оптимальну сегментацію часових рядів з більш ніж тисячею членів.)

Сегментація за допомогою динамічного програмування

Середньоквадратична помилка на сегменті як функція від параметрів моделі (The segment error function)

Повне визначення вартості сегментації вимагає специфікації середньоквадратичної помилки на сегменті (*segment error*) $d_{s,t}$ для кожної пари (s, t) таких, що $0 \leq s < t \leq T$. Використовуємо лінійну регресію, щоб представити досить загальну сім'ю середньоквадратичних помилок на сегменті як функцій (*segment error function*). Припустимо, що часовий ряд $x_1, x_2, \dots, x_T$ породжений процесом вигляду: $x_\tau = a^1 u_\tau^1 + a^2 u_\tau^2 + \dots + a^M u_\tau^M + e_\tau$, де $u_\tau^1, u_\tau^2, \dots, u_\tau^M$ - відповідні вхідні змінні, e_τ - похибки (залишки) з нульовим середнім та параметри моделі $a^1, a^2, \dots, a^M$ залишаються постійними в межах кожного сегменту, але змінюються між сегментами. Іншими словами, ми припускаємо, що в межах k -го сегмента часовий ряд задовольняє співвідношення (для $\tau = t_{k-1} + 1, t_{k-1} + 2, \dots, t_k$)

$$x_\tau = a_k^1 u_\tau^1 + a_k^2 u_\tau^2 + \dots + a_k^M u_\tau^M + e_\tau,$$

та a_k^m (для $m=1, 2, \dots, M$) - відповідні коефіцієнти.

Ми визначаємо $d_{s,t}$ як *похибку* регресійної моделі, підбраної до вибірок $x_s, x_{s+1}, \dots, x_t$. Якщо вважати $[s, t]$ сегментом, то оцінка x_τ для $\tau \in [s, t]$ має вигляд

$$\hat{x}_\tau = \hat{a}_{s,t}^1 u_\tau^1 + \hat{a}_{s,t}^2 u_\tau^2 + \dots + \hat{a}_{s,t}^M u_\tau^M, \quad (4)$$

де $\hat{a}_{s,t}^m$ – оцінки методом найменших квадратів, отримані з $x_{s+1}, x_{s+2}, \dots, x_t$ (отже, $\hat{x}_\tau$ залежить від s і t). Тоді похибка сегмента (segment error) має вигляд

$$d_{s,t} = \sum_{\tau=s}^t (x_\tau - \hat{x}_\tau)^2, \quad (5)$$

де $\hat{x}_\tau$ задається рівнянням 4. Наведемо наш конкретний приклад (лінійний випадок):

Лінійна регресія: $M=2$, $u_t^1 = 1$, $u_t^2 = t$ і модель має вигляд (6)

$$x_t = a_k^1 \cdot 1 + a_k^2 \cdot t + e_t.$$

В подальшому коефіцієнти M , u_t^1 і u_t^2 можуть бути іншими.

Коли s і t є істинними межами сегментів, $d_{s,t}$ відображає лише шум даних (наприклад, якщо e_τ дорівнює 0 для $\tau = s, s+1, \dots, t$, то $d_{s,t}$ також дорівнюватиме нулю); коли $[s, t]$ охоплює два істинні сегменти, то $d_{s,t}$ також містить систематичну похибку (оскільки ми намагаємось підігнати два регресійні співвідношення за допомогою одного набору коефіцієнтів). Отже, менші значення $d_{s,t}$ асоціюються з істинними відрізками.

Що, якщо $[s, t]$ є підмножиною істинного сегмента? Загалом, вартість сегментації завжди можна зменшити, розбивши істинні сегменти на підсегменти. Насправді, ми можемо зробити сильніше твердження.

Позначимо через $\hat{J}(K)$ *оптимальну* вартість сегментації (optimal segmentation cost), отриману за допомогою сегментації з K сегментами. Іншими словами, ми задаємо

$$\hat{J}(K) = \min_{t \in T_K} J(t).$$

Легко перевірити, що $K > K'$ означає $\hat{J}(K) \leq \hat{J}(K')$ [для кожного $d_{s,t}$ вигляду (Рівняння 5)]. Як (екстремальний) випадок, зауважимо, що $\hat{J}(T) = 0$; це виходить з тривіальної сегментації $(0, 1, 2, \dots, T)$, де кожен відрізок містить одну вибірку. Але така сегментація не дає жодної інформації про структуру часового ряду. Отже, ми повинні остерігатися використання "забагато" сегментів. Це проблема визначення порядку сегментації (*segmentation order*) – знаходження кількості сегментів, і вона є окремим випадком вибору порядку моделі (*model order selection*), проблеми, яка широко вивчається в статистичній літературі.

Алгоритм динамічного програмування

Тепер ми подивимось власне алгоритм сегментації. Мотивація для цього алгоритму полягає у наступному. Розглянемо оптимальну сегментацію $x_1, x_2, \dots, x_t$ яка містить K сегментів (відрізків), і припустимо, що її останній відрізок – $[s+1, t]$.

Тоді решта $K-1$ відрізків утворюють оптимальну сегментацію $x_1, x_2, \dots, x_s$.

Точніше, якщо $c_t^{(k)}$ - мінімальна вартість розбиття $x_1, x_2, \dots, x_t$ на k сегментів, то маємо

$$c_t^{(k)} = c_s^{(k-1)} + d_{s+1,t}.$$

Це типовий підхід динамічного програмування і може бути реалізований за допомогою наступного алгоритму.

Після завершення, алгоритм обчислює оптимальну вартість сегментації

$$c_{T,K} = \hat{J}(K) = \min_{t \in T_K} J(t),$$

і, повертаючись назад, оптимальну сегментацію

$$\hat{t}^{(K)} = (t_0^{(K)}, t_1^{(K)}, \dots, t_K^{(K)});$$

ці величини були обчислено для $K = 1, 2, \dots, K_{max}$, тобто рекурсивно розв'язано низку задач мінімізації. [19]

Розділ 2

Практична частина

1. Програмне забезпечення

1.1. Використані технології

На даний момент Python є найтипівішою мовою програмування для аналізу даних та машинного навчання, тому мій вибір пав саме на неї. Python є потужною та відносно легкою до сприйняття мовою. На відміну від більш складних мов, вона зосереджена на читабельності, а її загальна популярність у технічній сфері означає, що багато програмістів вже знайомі з нею. Також, на відміну від R, реалізувати код для цієї роботи було простіше на Python, тому це також стало одним з вагомих чинників.

Python є надзвичайно універсальною мовою; вона має величезний вибір бібліотек ресурсів, придатних для вирішення різноманітних завдань з аналізу даних.

Для реалізації поставленого завдання, я використовувала PyCharm як програмне середовище, в якому я встановила і потрібні для розробки бібліотеки: NumPy, Matplotlib та Pandas. Бібліотеки NumPy та Pandas чудово підходять для оптимізації високообчислювальних завдань, а також для підтримки загальних маніпуляцій з даними. Matplotlib використовується для візуалізації даних. В нашому випадку використовуємо Pandas для зчитування реальних даних з датасету в форматі csv та записування їх в форматі DataFrame.

1.2. Опис та збір даних

Для початку, щоб перевірити роботу алгоритму, я використовувала невелику кількість точок в часовому ряді, які були з лінійною регресією та без шумів. Далі, за допомогою генератора випадкових чисел та інших функцій

(описано в пункті 3 цього розділу) створено великий випадковий часовий ряд X з лінійною регресією та випадковим шумом, який має нормальний розподіл.

Джерелом реальних даних для цієї роботи є джерело [31] – сайт Kaggle. Kaggle — це платформа для змагань з аналітики та передбачувального моделювання, в рамках якого статистики та добувачі даних конкурують у створенні найкращих моделей для прогнозування та опису даних, запропонованих компаніями або користувачами. Тому, крім цього, Kaggle є ще і онлайн-платформою для обміну та отримання даних на різну тематику.

Для демонстрації роботи нашого алгоритму ми подивимось дані щодо продажів товарів (Sample Sales Data) за 2003, 2004 та 2005 роки. А саме щодо продажів моделей машинок, літаків та мотоциклів (Vintage Cars, Trucks and Buses, Classic Cars, Planes, Motorcycles) в магазинах іграшок, сувенірів та колекційних магазинах різних країн. В датасеті містяться такі дані: дані про продажі, інформацію про замовлення, продажі, клієнтів, доставку тощо. Ці дані були створені для аналітики роздрібної торгівлі, а саме для сегментації, клієнтської аналітики, кластеризації та іншого. Дані мають такий вигляд:

```
ORDERNUMBER,QUANTITYORDERED,PRICEEACH,ORDERLINENUMBER,SALES,ORDERDATE,STATUS,QTR_ID,MONTH_ID,YEAR  
ID,PRODUCTLINE,MSRP,PRODUCTCODE,CUSTOMERNAME,PHONE,ADDRESSLINE1,ADDRESSLINE2,CITY,STATE,POSTALCODE  
,COUNTRY,TERRITORY,CONTACTLASTNAME,CONTACTFIRSTNAME,DEALSIZE  
10107,30,95.7,2,2871,2/24/2003 0:00,Shipped,1,2,2003,Motorcycles,95,S10_1678,Land of Toys  
Inc.,2125557818,897 Long Airport Avenue,,NYC,NY,10022,USA,NA,Yu,Kwai,Small
```

Для нашої роботи використовуємо здебільшого колонку SALES.

2. Опис алгоритму

Для реалізації алгоритму сегментації за допомогою динамічного програмування ми використовуємо теорію з розділу 1, особливо з пунктів 1.2. і 2.5.

Для початку, створюємо функцію **linear_regression_cost()**. Вона обчислює вартість лінійної регресії для заданого часового ряду (сегмента) X . Якщо більш

детально, то також вона створює матрицю *X_matrix* з одиницями та індексами, обчислює коефіцієнти лінійної регресії *A*, обчислює залишки *residuals* та підсумовує квадрати залишків для отримання вартості. Цю функцію ми будемо використовувати для реалізації алгоритму сегментації часового ряду. Покроково це виглядає так:

- Обчислюємо кількість спостережень *n* у сегменті *X*.
- Далі формується матриця *X_matrix*, яка містить стовпчик одиниць та стовпчик індексів (від 0 до *n*-1).
- Вектор *y* містить значення сегмента *X*.
- Обчислюємо коефіцієнти лінійної регресії *A* за допомогою псевдооберненої матриці (`np.linalg.pinv()`).
- Обчислюємо залишки *residuals* як різницю між вектором *y* та оцінками, отриманими за допомогою *X_matrix* та *A*.
- Вартість регресії обчислюється як сума квадратів залишків (`np.sum(residuals ** 2)`).
- Функція повертає обчислену вартість.

Створюємо основну функцію – **`segment_time_series()`**. Вона реалізовує алгоритм сегментації часового ряду. На основі отриманих сегментацій та кількості сегментів (*K*) в них, функція реалізовує ще одну важливу деталь: визначення “істинної” кількості сегментів за байєсівським інформаційним критерієм (BIC). Аргументами та параметрами (те, що функція приймає на вхід) є часовий ряд *X* та максимальна кількість сегментів *max_segments*. Функція повертає список оптимальних сегментацій для кожного значення *K* від 1 до *max_segments*, а також результати BIC – найоптимальніша сегментація та інформація про неї, за цим критерієм. Саме тут ми використовуємо функцію створену раніше – `linear_regression_cost()`. Для чого саме вона тут потрібна?

1. Обчислення вартості сегментів: алгоритм сегментації намагається знайти точки, в яких сума квадратів залишків (тобто вартість) лінійної регресії кожного сегмента буде мінімальною. Функція `linear_regression_cost()` підраховує саме цю вартість для заданого сегмента часового ряду.

2. Динамічне програмування: алгоритм `segment_time_series()` використовує динамічне програмування для пошуку оптимальних точок сегментації. На кожному кроці він обчислює мінімальну вартість сегментації до поточної точки. Ця мінімальна вартість безпосередньо залежить від вартості лінійної регресії для кожного сегмента, обчисленої за допомогою функції `linear_regression_cost()`.

3. Визначення оптимальної сегментації: після обчислення мініимальної вартості сегментації для кожного можливого числа сегментів, алгоритм визначає оптимальну сегментацію, тобто ту, що має найменшу загальну вартість. Ця вартість розраховується на основі результатів, отриманих з `linear_regression_cost()`.

Алгоритм **`segment_time_series()`** складається з декількох частин: ініціалізація для $K=1$, динамічне програмування для $K>1$ та знаходження оптимальної кількості сегментів за BIC.

Ініціалізація для $K=1$: обчислюємо вартість лінійної регресії для всього часового ряду та зберігаємо її в `c[T]`. Результат (вартість та сегментація) додається до списку `optimal_segmentations`. Покроково це виглядає так:

- Ініціалізуємо список `optimal_segmentations`, який буде зберігати результати для кожного значення K .

- Ініціалізуємо крок $K=1$: обчислюємо вартість лінійної регресії для всього часового ряду `X[:T]` за допомогою `linear_regression_cost()` та додаємо до `optimal_segmentations`.

Динамічне програмування для $K>1$: використовуємо алгоритм динамічного програмування для знаходження оптимальної сегментації. Для кожного s (потенційної точки сегментації) обчислюється мінімальна вартість сегментації до цієї точки та зберігається в `c_new[s]`. Також зберігається індекс t , який відповідає оптимальній точці сегментації перед s . Після обчислення всіх `c_new` та `z_new`, оновлюємо `c` та `z` для наступної ітерації. Покроково це виглядає так:

- Ініціалізуємо масиви c_new та z_new , які будуть зберігати мінімальну вартість та індекси точок сегментації для поточного K .
- Для кожної потенційної точки сегментації s обчислюється мінімальна вартість сегментації до цієї точки. Це робиться шляхом перебору всіх можливих точок сегментації $t < s$ та обчислення вартості як $c[t] + \text{linear_regression_cost}(X[t:s])$.
- Якщо знайдена вартість менша за поточний мінімум, вона оновлюється, а також оновлюється індекс $best_t$, який вказує на оптимальну точку сегментації перед s .
- Після заповнення c_new та z_new , вони оновлюють c та z для наступної ітерації.

Наступний крок цієї частини – знаходження оптимальної сегментації: на основі масиву z , який зберігає індекси оптимальних точок сегментації, будується фінальна сегментація для поточного K .

Покроково це виглядає так:

- Після завершення циклу для K , знаходимо оптимальну сегментацію для поточного K на основі масиву індексів z .
- Ініціалізуємо список `segmentation` з першим елементом 0.
- Потім, починаючи з останньої точки T , крок за кроком додаємо індекси оптимальних точок сегментації, збережені в z .
- Наприкінці до `segmentation` додаємо останню точку T .
- Оптимальна сегментація, разом із її вартістю, додається до `optimal_segmentations`.

Останнім етапом є знаходження оптимальної кількості сегментів за ВІС та виведення результатів функції. Покроково, ми створюємо список, і для кожного проходу по `optimal_segmentations` ми обчислюємо значення ВІС для поточної кількості сегментів K за формулою (див. Вибір порядку сегментації в 1.2. розділу 1), та додаємо до створеного на початку списку. Виводимо результати.

3. Результати та їх аналіз

Після реалізації потрібного алгоритму робимо кроки для отримання результатів:

Для перевірки роботи алгоритму використовувалось декілька видів задання даних:

1. Задання часового ряду просто як масив X .
2. Створення випадкового часового ряду за допомогою генератора випадкових чисел, і подальшого розбиття на сегменти.

Покроково це виглядає так:

- Задаємо `seed` для генератора випадкових чисел `np.random.seed(0)`. Це означає, що кожного разу, коли ми запустимо цей код, він буде генерувати однакові випадкові числа. Цим ми зможемо повторити відтворювані результати.

- Задаємо `slope (0.5)` та `intercept (2.0)`, які визначають лінійність у даних. Лінійність означає, що значення даних будуть збільшуватися на сталу величину (`slope`) при кожному збільшенні часової позначки на 1. Початкове значення тренду задається `intercept`. Наприклад, якщо `slope=0.5` та `intercept=2.0`, то значення даних будуть 2.0, 2.5, 3.0, 3.5 і так далі.

- За допомогою генератора випадкових чисел, ми створюємо часовий ряд X шляхом застосування лінійності (`slope * i + intercept`) та додавання випадкового шуму із нормальним розподілом з середнім 0 та стандартним відхиленням 5 (`np.random.normal(0, 5)`) для 100 точок. i послідовно приймає значення від 0 до 99 (100 значень) у кожній ітерації генераторного виразу. Випадковий шум із нормальним розподілом (з середнім 0 та стандартним відхиленням 5) імітує реальні коливання, які можуть бути присутні у часовому ряді. Якби ми не додавали `np.random.normal(0, 5)`, то ряд X містив би лише значення ідеальної лінійної функції без жодних відхилень або шуму. Але завдяки додаванню випадкових значень з нормального розподілу, кожне значення в X відхиляється від ідеальної лінії на деяку випадкову величину.

3. Задання реальних даних. Покроково це виглядає так:

- Зчитуємо дані з csv файлу, в якому лежить наш датасет продажів. І записуємо їх в змінну df, формат якої – DataFrame. DataFrame - це двовимірна табличноподібна структура даних, де рядки представляють спостереження, а стовпці - змінні.

- З датафрейму df вибирається стовпець SALES за допомогою індексації df['SALES']. Метод tolist() перетворює вибрані дані зі стовпця SALES на список. Отриманий список присвоюємо змінній X.

Як покроково ми розбиваємо часовий ряд на сегменти за допомогою нашого алгоритму:

- Викликаємо функцію segment_time_series(X, max_segments=5), яка виконує сегментацію часового ряду X з максимальною кількістю сегментів, рівною 5.

- Функція повертає список optimal_segmentations, який містить кортежі з вартістю сегментації та оптимальною сегментацією для кожного значення K від 1 до 5. Також повертає оптимальну сегментацію та інформацію про неї, визначену за BIC.

Наступним етапом є побудова графіку: використовуючи бібліотеку matplotlib.pyplot, будуємо графік часового ряду X. Додатково на графіку рисуємо вертикальні лінії, що позначають оптимальні точки сегментації для кожного значення K. Покроково це виглядає так:

- Створюємо новий графік із заданим розміром 14x8 (figsize=(14, 8)).
- Будуємо графік часового ряду X за допомогою plt.plot(X, ...). Використовуємо маркери у вигляді кружечків та суцільну лінію.

- Створюємо цикл для сортування точок в порядку зростання – він проходить по всім оптимальним сегментаціям, отриманих у optimal_segmentations і сортує точки сегментації за допомогою функції sort().

- Записуємо в змінну best_segmentation найоптимальнішу сегментацію (останню)

- Далі проходимо по всім внутрішнім точкам цієї найоптимальнішої сегментації (крім першої та останньої точки):

-- plt.plot(seg_point, X[seg_point], marker='o', markersize=8, color='red') –

додаємо червоні кружечки у точках сегментації на графіку.

-- plt.axvline(x=seg_point, color='black', linestyle='--') - додаємо вертикальні чорні пунктирні лінії, що проходять через точки сегментації.

- Щоб графік гарно виглядав, додаємо підписи по осям X і Y: plt.xlabel('Часова відмітка') - по X, plt.ylabel('Значення') – по Y, та додамо заголовок: plt.title('Сегментація великого часового ряду').

- Додаємо легенду до графіку та сітку для кращої візуалізації результатів.

- plt.show() - відображаємо наш побудований графік.

Окрім графіку, для кожної оптимальної сегментації з optimal_segmentations виводимо в консоль інформацію про вартість сегментації cost та саму сегментацію segmentation у вигляді списку. А також оптимальну сегментацію, яку визначив алгоритм за критерієм BIC.

Тепер подивимось безпосередньо результати запуску нашого алгоритму (та всього коду). Проаналізуємо отримані результати:

Для моєї невеликої вибірки маємо такі результати:

```
x = [1, 2, 3, 4, 5, 6, 7, 10, 15, 20, 25, 30, 35, 40, 45, 50, 55, 60, 65, 70]
```

Оптимальна кількість сегментів за BIC: 2

Вартість для оптимальної сегментації: 5.977642813118793e-27

Оптимальна сегментація: [0, 7, 20]

Cost: 496.1894736842104, Segmentation: [0, 20]

Cost: 5.977642813118793e-27, Segmentation: [0, 7, 20]

Cost: 5.611611353096244e-27, Segmentation: [0, 15, 18, 20]

Cost: 3.3901790439938745e-27, Segmentation: [0, 11, 14, 17, 20]

Cost: 1.1939902838585777e-27, Segmentation: [0, 10, 11, 14, 17, 20]

І графік для найоптимальнішої сегментації за критерієм BIC (рисунок 2).

За цим графіком вже можна зробити висновок, що алгоритм працює правильно. Зміна коефіцієнтів лінійної регресії відбувається тільки один раз, в

точці 7. Як раз в цій точці, за критерієм ВІС, наш алгоритм розділив часовий ряд на два сегменти. Тобто часовий ряд має дві фази: поступове зростання на початку та більш стрімке зростання наприкінці, і сегментація успішно виділила ці фази.

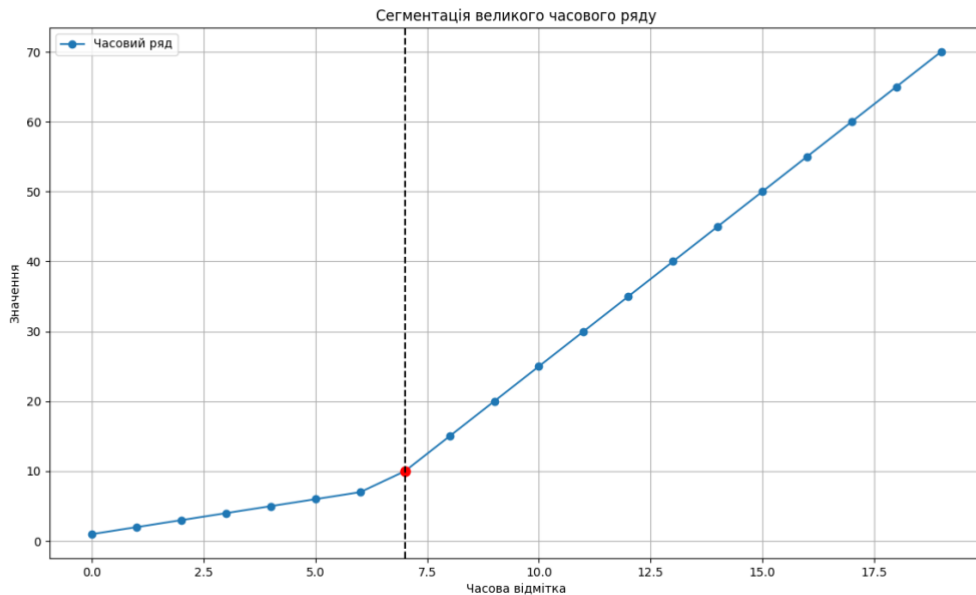


Рис. 2

Також якщо порівнювати всі п'ять вартостей сегментацій, вони зменшуються з кожною з них, як і має бути. При цьому зі збільшенням кількості сегментів, вартість сегментації (сумарна похибка) зменшується. Це свідчить про покращення якості апроксимації при зростанні кількості сегментів від 2 до 5.

Для чого взагалі потрібно знати вартість сегментації та чому вона має зменшуватися з кожною сегментацією?

- Алгоритм сегментації намагається знайти оптимальні точки сегментації, які мінімізують загальну вартість лінійної регресії для кожного сегмента. Тобто він шукає точки, в яких сума квадратів залишків (вартість) для сегментів буде мінімальною.

- Після обчислення мінімальної вартості сегментації для кожного можливого числа сегментів, алгоритм визначає оптимальну сегментацію, тобто ту, що має найменшу загальну вартість.

Отже, по цьому, хоч і невеликому часовому ряду, можна припустити, що алгоритм працює правильно.

Тепер подивимось результати для згенерованого часового ряду за допомогою генератора випадкових чисел.

Маємо такі результати:

Оптимальна кількість сегментів за BIC: 3

Вартість для оптимальної сегментації: 2128.882629877363

Оптимальна сегментація: [0, 67, 5, 100]

Cost: 2520.777133671531, Segmentation: [0, 100]

Cost: 2128.882629877363, Segmentation: [0, 5, 67, 100]

Cost: 1973.3055514263742, Segmentation: [0, 34, 42, 44, 100]

Cost: 1827.7971885571824, Segmentation: [0, 23, 25, 32, 38, 100]

Cost: 1696.0792218113725, Segmentation: [0, 21, 24, 26, 32, 38, 100]

Вартість сегментацій зменшується, що свідчить, що алгоритм працює правильно. Подивимось на графік сегментації з найменшою вартістю: [0, 21, 24, 26, 32, 38, 100]. Нарисуємо два графіки, для більш гарної видимості результатів: один без пунктирних ліній – рисунок 3 (тобто ми закоментували рядок `plt.axvline(x=seg_point, color='black', linestyle='--')`), а інший – з пунктирами (рисунок 4).

Часовий ряд вже не є таким лінійним і простим як минулий. Видно і стрибки, і зони, де дані коливаються слабо. Генератор випадкових чисел створив нам ряд, який має складну структуру з багатьма різкими змінами значень, що вказує на нестабільність процесу, який він описує.

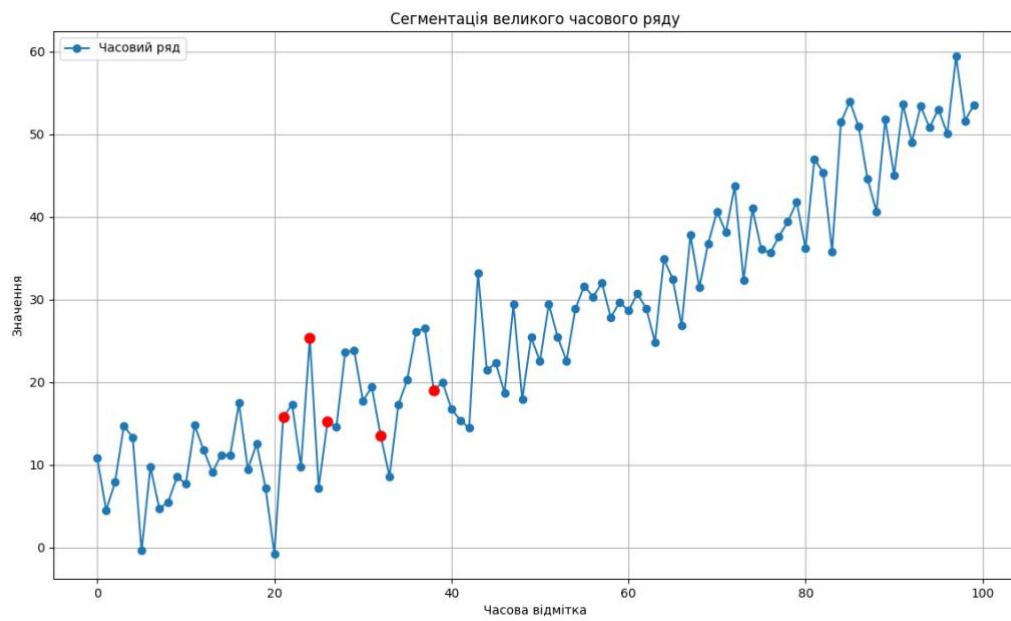


Рис. 3

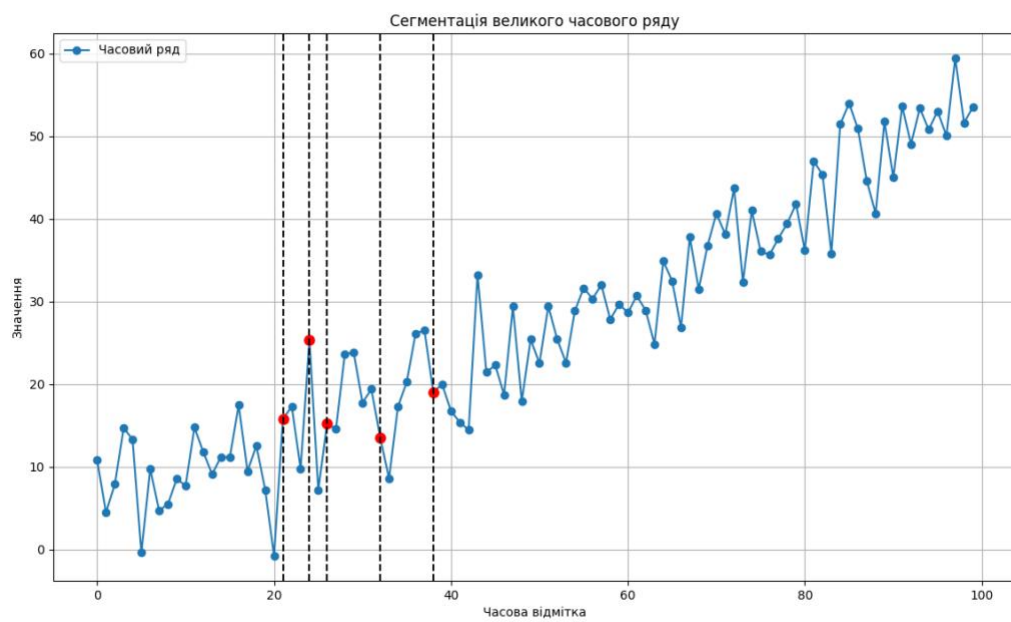


Рис. 4

Наш алгоритм успішно виділив кілька основних сегментів часового ряду, які

відрізняються своєю характеристикою та поведінкою. Це може дати нам можливість проаналізувати динаміку процесу в кожному сегменті.

Згідно з результатами, сегментація з найменшою вартістю має шість сегментів: [0, 21, 24, 26, 32, 38, 100]. Аналіз значень вартості сегментації (Cost) для різної кількості сегментів дозволяє оцінити, наскільки складна структура часового ряду і яка кількість сегментів оптимально її відображає. Шість сегментів здаються хорошим компромісом між складністю моделі та якістю апроксимації. Але найоптимальніша сегментація за критерієм BIC, в цьому випадку, не є тою, у якої найменша вартість. Критерій вибрав найоптимальнішу сегментацію з двома сегментами: [0, 5, 67, 100], графік наведено нижче (рисунк 5). Використання BIC дозволяє знайти баланс між якістю підгонки та складністю моделі, що допомагає уникнути надмірної підгонки. Це забезпечує більш обґрунтований вибір оптимальної кількості сегментів для представлення даних.

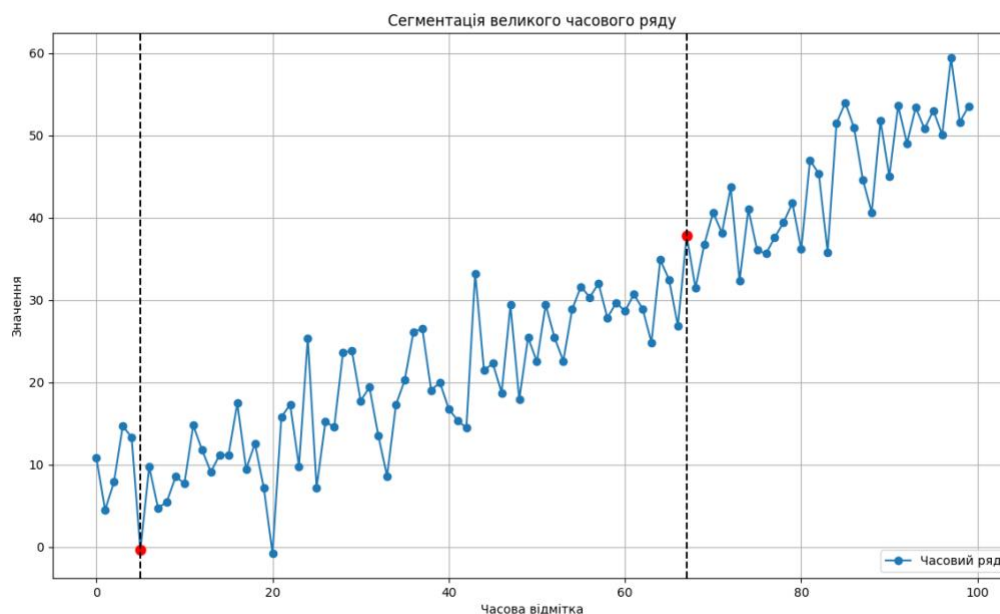


Рис. 5

Результат на реальних даних – часовому ряду продажів.

Маємо такі результати:

Оптимальна кількість сегментів за BIC: 6

Вартість для оптимальної сегментації: 2511588771.987774

Оптимальна сегментація: [0, 675, 600, 501, 334, 204, 742]

Cost: 3015645376.322877, Segmentation: [0, 742]

Cost: 2879556190.1048856, Segmentation: [0, 25, 45, 742]

Cost: 2750671714.1237564, Segmentation: [0, 45, 78, 334, 742]

Cost: 2651322490.6226625, Segmentation: [0, 204, 501, 600, 675, 742]

Cost: 2511588771.987774, Segmentation: [0, 204, 334, 501, 600, 675, 742]

Вартість сегментації зменшується, що свідчить, що алгоритм працює правильно. Подивимось на графік найоптимальнішої сегментації (рис. 6):

Cost: 2511588771.987774, Segmentation: [0, 204, 334, 501, 600, 675, 742]. Тут в нас співпала сегментація з найменшою вартістю, і найоптимальніша сегментація за критерієм BIC.

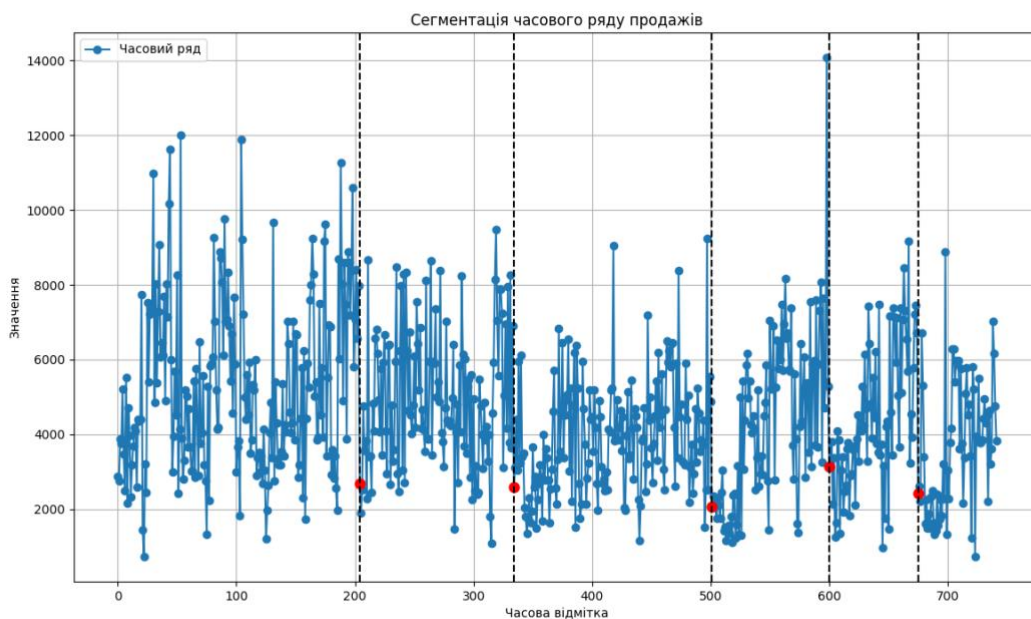


Рис. 6

Візуально, наша оптимальна сегментація добре розділила часовий ряд продажів на сегменти. Збільшимо кількість сегментів до 11:

Оптимальна кількість сегментів за BIC: 11

Вартість для оптимальної сегментації: 2114899104.0589623

Оптимальна сегментація: [0, 675, 600, 598, 569, 501, 350, 317, 273, 204, 186, 742]

Cost: 3015645376.322877, Segmentation: [0, 742]

Cost: 2879556190.1048856, Segmentation: [0, 25, 45, 742]

Cost: 2750671714.1237564, Segmentation: [0, 45, 78, 334, 742]

Cost: 2651322490.6226625, Segmentation: [0, 204, 501, 600, 675, 742]

Cost: 2511588771.987774, Segmentation: [0, 204, 334, 501, 600, 675, 742]

Cost: 2412094991.367682,

Segmentation: [0, 204, 318, 334, 501, 600, 675, 742]

Cost: 2337413616.521395,

Segmentation: [0, 204, 273, 317, 350, 501, 600, 675, 742]

Cost: 2254326541.7117977,

Segmentation: [0, 186, 204, 318, 350, 501, 569, 600, 675, 742]

Cost: 2183021821.7704587,

Segmentation: [0, 186, 204, 273, 318, 350, 501, 569, 600, 675, 742]

Cost: 2114899104.0589623,

Segmentation: [0, 186, 204, 273, 317, 350, 501, 569, 598, 600, 675, 742]

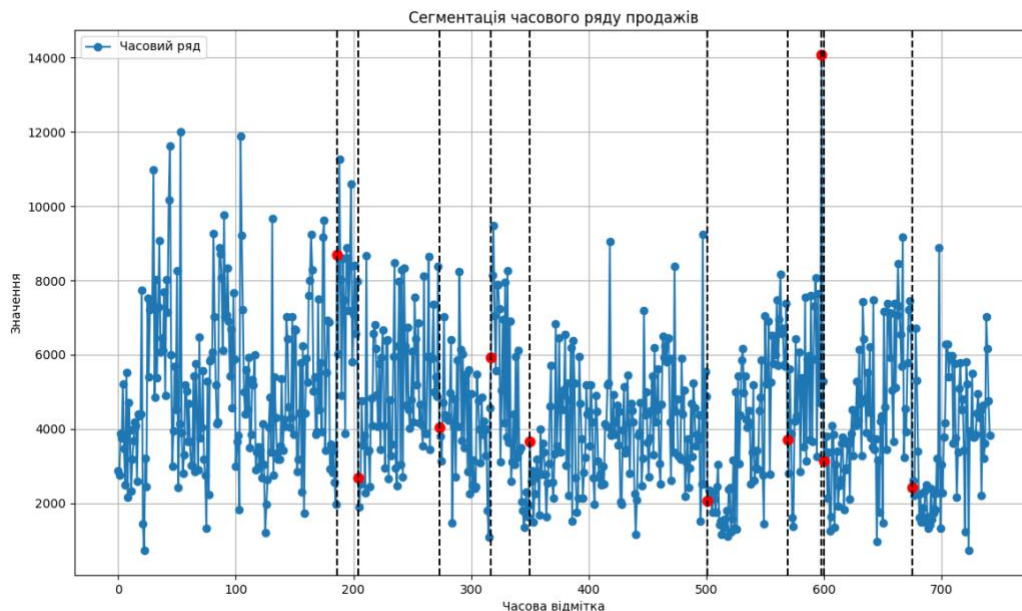


Рис. 7

Алгоритм працює правильно і досить добре. Бачимо, що сегментація добре розділила наш часовий ряд продажів. Кожен сегмент відрізняється один від іншого: наприклад, алгоритм побачив нетипову поведінку для шматочка часового ряду від 186 до 204 і відділив його від першого сегменту точкою сегментації. Аналогічно і далі. При нетиповій поведінці ряду відбувається відділення шматочків до окремих сегментів.

Чому тоді на рис. 7 алгоритм не відділив стрибки в першому сегменті $[0, 186]$ як нетиповість ряду? Насправді, якщо подивитися на сегментації з більшою вартістю:

Cost: 2879556190.1048856, Segmentation: $[0, 25, 45, 742]$

Cost: 2750671714.1237564, Segmentation: $[0, 45, 78, 334, 742]$

які не є найоптимальнішими, то там відбувалося відділення тих шматочків на окремі сегменти. Але при подальшому пошуку оптимальної сегментації, у випадку BIC, додавання цих сегментів не покращило б значно якість підгонки моделі до даних, враховуючи збільшення складності моделі.

Простими словами, роботу BIC для сегментації можна пояснити так.

Байєсівський інформаційний критерій (BIC) прагне знайти оптимальний баланс між якістю підгонки моделі та її складністю. Для цього він накладає штраф на складніші моделі (з більшою кількістю параметрів). Штраф зростає зі збільшенням кількості параметрів і розміру даних.

У випадку з першим великим сегментом, коливання даних у цьому діапазоні, ймовірно, не було достатньо високим, щоб виправдати додавання додаткових сегментів.

Найоптимальніша сегментація за критерієм BIC співпала з сегментацією з найменшою вартістю (сумарною похибкою). Це відбулося тому що покращення підгонки моделі до даних при додаванні 11-го сегмента було настільки значним, що воно перевершило збільшення штрафу за складність, пов'язаного з додаванням нових параметрів. Тому 11-сегментна модель мала найменше значення BIC серед усіх розглянутих моделей і була визнана оптимальною. Критерій BIC оцінює кожну модель окремо та обирає ту, яка забезпечує найкращий баланс між якістю підгонки та простотою.

Дивлячись на сегменти, цікавим є випадок сегменту 9 – [598, 600]. На рис. 7 його майже не видно, оскільки він є найменшим – всього 2 елементи.

Але цьому також є пояснення. Точка цього сегменту є найбільшою в усьому часовому ряді – тобто це є найбільшою виручкою в певний момент часу. Тому алгоритм відділив її в окремий сегмент.

Загалом, дивлячись на графік на рис. 7, можна сказати, що сегменти 1, 6 та 11 (1: [0, 186], 6: [350, 501], 11: [675, 742]) мають багато стрибків, піків та спадів. Це може бути спричинено коливанням попиту, сезонністю, або різноманітними маркетинговими акціями.

На графіку, для сегментів 2, 4, 5 ([186, 204], [273, 317], [317, 350]) можна побачити період спадання попиту. В 4 сегменті ряд має більш плавне спадання, на відміну від 2 і 5 – там воно доволі різке. В сегменті 2, ряд починає різкий спад з найвищих значень продажів, при порівнянні з сегментами 4 та 5.

Сегменти 3, 7, 8, 10 ([204, 273], [501, 569], [569, 598], [600, 675]) свідчать про зростання попиту на товари (нагадаємо: *Розділ 2, 1.2. Опис та збір даних*, що ми дивимось продажі моделей машинок, літаків та мотоциклів).

Зростання в випадку сегментів 7 та 10 є доволі стрибковим, інші – більш плавні. Загалом, в нашому випадку, зростання є більш плавним за спадання, тому з цього можна зробити висновок, що така категорія як моделі машинок дуже швидко втрачають популярність (наприклад, якщо компанія не виготовляє нових моделей), а набувають її доволі довго.

Також, важливо зауважити, що для всіх сегментацій (дані продажів; часовий ряд, згенерований генератором випадкових чисел; невеликий ряд, створений вручну) зі збільшенням кількості сегментів, вартість сегментації (сумарна похибка) зменшується.

Висновки

У роботі ми реалізували алгоритми сегментації даних за допомогою динамічного програмування, спираючись на теоретичну частину та відповідні функції в бібліотеках NumPy на мові Python. Також нарисували графіки за допомогою бібліотеки Matplotlib, які були створені на основі нашого алгоритму, що сегментував дані.

Ми перевірили ефективність роботи алгоритму на малих даних - масиві, який був написаний вручну, на ряді, який був згенерований генератором випадкових чисел з лінійною регресією та випадковою похибкою, та на реальних даних, які відображали продажі товарів за певний період.

Результати даної роботи демонструють ефективність та перспективність підходу, заснованого на методах динамічного програмування, для задачі сегментації експериментальних кривих та часових рядів. Розроблений алгоритм дозволяє знаходити оптимальний розподіл вихідних даних на сегменти, в межах яких дані є однорідними за певним критерієм. При цьому, застосування байєсівського інформаційного критерію забезпечує обґрунтований вибір оптимальної кількості сегментів, знаходячи баланс між якістю апроксимації даних та складністю моделі.

Візуалізація результатів у вигляді графіків з позначеними точками сегментації полегшує інтерпретацію отриманих сегментів та виявлення особливостей поведінки часових рядів. Запропонований підхід можна застосовувати в подальшому в різних задачах аналізу даних, де необхідно ідентифікувати зміни режимів чи тенденцій, наприклад, в гідрології, моніторингу навколишнього середовища, фінансовій аналітиці тощо. Наприклад, в нашому випадку, отримана оптимальна сегментація для ряду продажів може бути використана для подальшого аналізу тенденцій продажів, прогнозування, виявлення аномалій чи планування маркетингових стратегій на основі виявлених сегментів.

Подальші дослідження можуть бути спрямовані на розширення функціональності методу, наприклад, для роботи з даними, які мають не тільки лінійну регресію.

Список використаних джерел

- [1] Akaike H (1974) *A new look at the statistical model identification*. IEEE Trans Automat Control 19:716–723
- [2] Auger IE, Lawrence CE (1989) *Algorithms for the optimal identification of segment neighborhoods*. Bull Math Biol 51:39–54
- [3] Bellman R (1960) *Dynamic programming*. M.: Izdatelstvo inostrannoy literatury
- [4] Bellman R (1961) *On the approximation of curves by line segments using dynamic programming*. Commun ACM 4:284
- [5] Bertele U., Brioshi F. (1972) *Nonserial dynamic programming*. N. Y.: Academic Press.
- [6] Braun JV, Braun RK, Mueller H-G (2000) *Multiple changepoint fitting via quasilielihood, with application to DNA sequence segmentation*. Biometrika 87:301–314
- [7] Braun JV, Mueller H-G (1998) *Statistical methods for DNA sequence segmentation*. Stat Sci 13:142–162
- [8] Chen J, Gupta AK (1998) *Information criterion and change point problem for regular models*. Tech. Rep. No. 98-05, Department of Math. and Stat., Bowling Green State U., Ohio
- [9] Cormen T, Leiserson C, Rivest R, Stein C (2001) *Introduction to Algorithms*. The MIT Press. 2-nd edition, Chapter 15
- [10] Eddy, S. R. (2004) *What is dynamic programming?* Nature Biotechnology, 22, 909—910
- [11] Fortin V, Perreault L, Salas JD (2004) *Retrospective analysis and forecasting of streamflows using a shifting level model*. J Hydrol 296:135–163

- [12] Fox, Emily B. (2008) *An HDP-HMM for systems with state persistence*. Proceedings of the 25th international conference on Machine learning. ACM.
- [13] Himberg J, Korpiaho K, Mannila H, Tikanmaki J, Toivonen HTT (2001) *Time series segmentation for context recognition in mobile devices*. Proc. of ICDM 2001, pp 203–210
- [14] Hubert P (1997) *Change points in meteorological analysis*. In: Subba Rao T, Priestley MB, Lessi O (eds) *Applications of time series analysis in astronomy and meteorology*. Chapman and Hall, London
- [15] Hubert P (2000) *The segmentation procedure as a tool for discrete modeling of hydrometeorological regimes*. Stoch Env Res Risk Ass 14:297–304
- [16] Jackson B (2005) *An algorithm for optimal partitioning of data on an interval*. IEEE Signal Proc Lett 12:105–108
- [17] Kay SM (1998) *Fundamentals of statistical signal processing: detection theory*. Prentice-Hall, Englewood Cliffs
- [18] Kearns M, Mansour Y, Ng AY, Ron D (1997) *An experimental and theoretical comparison of model selection methods*. Machine Learn 27:7–50
- [19] Kehagias Ath, Nidelkou Ev, Petridis V (2005) *A dynamic programming segmentation procedure for hydrological and environmental time series*. Stoch Environ Res Risk Assess
- [20] Kehagias Ath (2004) *A hidden Markov model segmentation procedure for hydrological and environmental time series*. Stoch Environ Res Risk Assess 18:117–130
- [21] Keogh E (2001) *An online algorithm for segmenting time series*. In: Proceedings of the 2001 IEEE International Conference on Data Mining
- [22] Keogh E, Chu S, Hart D, Pazzani M (2003) *Segmenting time series: a survey and novel approach*, In: *Data mining in time series databases*. World Scientific Publishing Company, Singapore

- [23] Li Y., Zhao K., Hu T., Zhang X. (2019) *BEAST: A Bayesian Ensemble Algorithm for Change-Point Detection and Time Series Decomposition*. Remote Sensing of Environment.
- [24] Ninomiya Y (2003) *Asymptotic theory for change-point model and its application to model selection*. Tech. Report, Graduate School of Mathematics, Kyusho Univ., Japan
- [25] Nocedal, J.; Wright, S. J. (2006) *Numerical Optimization*, page 9. Springer.
- [26] Pavlidis T, Horowitz SL (1974) *Segmentation of Plane Curves*. IEEE Trans Comput 23:860–870
- [27] Priestley M.B. (1981) *Spectral Analysis and Time Series*. Academic Press, p. 375
- [28] Schwarz G (1978) *Estimating the dimension of a model*. Ann Stat 6:461–464
- [29] Yao Y-C (1988) *Estimating the number of change-points via Schwarz' criterion*. Stat Prob Lett 6:181–189
- [30]
https://neerc.ifmo.ru/wiki/index.php?title=Динамическое_программирование
- [31] <https://www.kaggle.com/datasets/kyanyoga/sample-sales-data/data>

Додаток 1

Код програми

```
# Імпорт бібліотек
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

# Функція, що обчислює вартість лінійної регресії для даного сегмента X
def linear_regression_cost(X):
    n = len(X)
    X_matrix = np.column_stack([np.ones(n), np.arange(n)])
    y = np.array(X)
    A = np.linalg.pinv(X_matrix.T @ X_matrix) @ X_matrix.T @ y
    residuals = y - X_matrix @ A
    cost = np.sum(residuals ** 2)

    return cost

# Функція, що реалізує алгоритм сегментації часового ряду
def segment_time_series(X, max_segments=10):
    T = len(X)
    optimal_segmentations = []

    # Ініціалізація для K=1
    c = np.zeros(T + 1)
    z = np.zeros(T + 1, dtype=int)
    for t in range(1, T + 1):
        c[t] = linear_regression_cost(X[:t])

    optimal_segmentations.append((c[T], [0, T]))

    # Динамічне програмування для K > 1
    for K in range(2, max_segments + 1):
        c_new = np.zeros(T + 1)
        z_new = np.zeros(T + 1, dtype=int)
        for s in range(1, T + 1):
            min_cost = float('inf')
            best_t = 0
            for t in range(s):
                cost = c[t] + linear_regression_cost(X[t:s])
                if cost < min_cost:
                    min_cost = cost
                    best_t = t
            c_new[s] = min_cost
            z_new[s] = best_t

        c, z = c_new, z_new
```

```

# Знаходження оптимальної сегментації
segmentation = [0]
s = T
for k in range(K):
    s = z[s]
    segmentation.append(s)
segmentation.append(T)

    optimal_segmentations.append((c[T], segmentation))

# Обчислення J(k) для кожної сегментації за формулою BIC
jk = []
for i, (cost, segmentation) in enumerate(optimal_segmentations):
    K = i + 1
    p = 2 * K
    bic = len(X) * np.log(cost / len(X)) + p * np.log(len(X))
    jk.append(bic)

# Знаходження оптимальної кількості сегментів за BIC
best_k = np.argmin(jk) + 1
best_cost, best_segmentation = optimal_segmentations[best_k - 1]

print(f"Оптимальна кількість сегментів за BIC: {best_k}")
print(f"Вартість для оптимальної сегментації: {best_cost}")
print(f"Оптимальна сегментація: {best_segmentation}")

return optimal_segmentations, best_k, best_cost, best_segmentation

# 2. Створення великого часового ряду з лінійною регресією
np.random.seed(0)
slope = 0.5
intercept = 2.0
X = [slope * i + intercept + np.random.normal(0, 5) for i in range(100)]

# 3. Задання реальних даних (продажі моделей)
df = pd.read_csv('sales1.csv')
X = df['SALES'].tolist()

# Розбиття часового ряду на сегменти
optimal_segmentations, best_k, best_cost, best_segmentation =
segment_time_series(X, max_segments=5)

# Побудова графіку
plt.figure(figsize=(14, 8))
plt.plot(X, label='Часовий ряд', marker='o', linestyle='-')

# сортування точок сегментації
for cost, segmentation in optimal_segmentations:
    segmentation.sort()

# (остання)сегментація з найменшою вартістю
# best_segmentation = optimal_segmentations[-1][1]

```

```

# рисування точок найоптимальнішої сегментації на графіку
for seg_point in best_segmentation[1:-1]:
    plt.plot(seg_point, X[seg_point], marker='o', markersize=8, color='red')
    plt.axvline(x=seg_point, color='black', linestyle='--')

plt.xlabel('Часова відмітка')
plt.ylabel('Значення')
plt.title('Сегментація часового ряду')
plt.legend()
plt.grid(True)
plt.show()

for cost, segmentation in optimal_segmentations:
    print(f"Cost: {cost}, Segmentation: {segmentation}")

```